PROGRAMMING ASSIGNMENT | 100 MARKS

Optimal Service-to-Server Assignment

Design, implement, visualize and evaluate a resource-constrained optimization algorithm.

Course	Optimization Methods
Assessment type	Individual practical programming assignment
Due date / time allowed	To be announced by the instructor

Objective

You are required to develop an interactive software application that assigns exactly 12 independent services to 4 heterogeneous servers. Each server has defined resource capacities, processing speed, energy characteristics, network latency, bandwidth and operating cost. Your application must search for a high-quality feasible allocation, never exceed server limits, and visually show how the algorithm progresses from candidate solutions to its best result.

A metaheuristic returns the best solution found, not necessarily a proven global optimum. You must make this distinction in the interface and report.

1. Learning outcomes

- Formulate service placement as a constrained, discrete optimization problem.
- Implement and explain an iterative optimization algorithm instead of merely producing a fixed or random allocation.
- Enforce resource and compatibility constraints for every accepted solution.
- Visualize algorithm iterations, live assignments, resource usage and objective improvement.
- Validate correctness, repeat experiments and interpret solution quality and computation time.

2. Problem definition and assumptions

- Each service is indivisible and must be assigned to one and only one server. Services may not be duplicated, split or omitted.
- All service requirements are reservations within one static placement scenario. Each server must satisfy CPU, RAM, storage, GPU-slot and maximum-service-count limits.
- The four servers have different throughputs, power characteristics and cost rates. GPU requirement is a hard compatibility/resource constraint.
- For the simplified processing-time model, each server processes assigned jobs sequentially in ascending service-ID order; throughput is the effective single-job processing rate. Network delay is assessed separately. All resource reservations remain allocated for the scenario.
- This is a simulation: no cloud accounts or physical servers are required. Algorithm wall-clock run time, simulated compute makespan and per-service network latency are three different quantities.
- Network model: services communicate with one representative client gateway. Each service transfers its listed data payload once over the assigned server link. RTT and bandwidth are fixed during a run; there is no simultaneous-transfer contention, queuing, packet loss or service-to-service traffic in the mandatory model.

3. Fixed initial dataset (editable inside the software)

Table 1. Server resource capacities and throughput

Server	vCPU	RAM (GB)	Disk (GB)	GPU slots	Max services	Speed (GI/s)
A	16	32	160	0	4	12

Server	vCPU	RAM (GB)	Disk (GB)	GPU slots	Max services	Speed (GI/s)
B	24	48	250	0	5	20
C	32	64	400	2	5	34
D	20	40	240	1	4	24

Table 2. Power consumption and price (paid per simulated busy second)

Server	Idle power (W)	Peak power (W)	Busy-time price (USD/s)
A	70	150	0.003
B	95	210	0.005
C	140	290	0.010
D	115	235	0.007

Table 3. Twelve independent services

Service	vCPU	RAM (GB)	Disk (GB)	Work (GI)	GPU slots
S01	4	8	30	90	0
S02	6	12	40	140	0
S03	5	10	50	120	0
S04	7	14	60	175	0
S05	3	6	25	65	0
S06	8	16	70	210	1
S07	5	10	45	130	0
S08	4	8	40	100	0
S09	6	12	55	165	1
S10	3	6	30	75	0
S11	7	14	75	190	1
S12	5	10	45	145	0

Table 4. Client-to-server network specifications (editable)

Server	Round-trip latency RTT (ms)	Available bandwidth (Mb/s)
A	18	500
B	30	800
C	60	1200
D	42	650

Table 5. Per-service incoming payload and maximum communication-latency SLA

Service	Payload (MB)	Max latency (ms)	Service	Payload (MB)	Max latency (ms)
S01	0.5	55	S07	1.2	65
S02	1.0	95	S08	0.5	75
S03	1.5	60	S09	1.0	80
S04	0.8	75	S10	0.3	95
S05	0.6	100	S11	2.5	105
S06	2.0	100	S12	0.8	80

All four tables must be preloaded, editable and resettable. GI = billions of instructions; GI/s = billions of instructions per second. MB is decimal megabytes, Mb/s is megabits per second (1 MB = 8 Mb). RTT is the base round-trip time from the client gateway to each server; no geographic/network measurement is required.

4. Mathematical formulation

Let x(i,j) ∈ {0,1}, where x(i,j)=1 if service i is assigned to server j and 0 otherwise. There are 12 services and 4 servers.

Mandatory feasibility constraints

- Unique assignment: for each service i, $\sum_j x(i,j) = 1$.
- For every server j: $\sum_i CPU(i) \cdot x(i,j) \le CPUcap(j)$.
- For every server j: $\sum_i RAM(i) \cdot x(i,j) \le RAMcap(j)$.
- For every server j: $\sum_i Disk(i) \cdot x(i,j) \le Diskcap(j)$.
- For every server j: $\sum_i GPU(i) \cdot x(i,j) \le GPUcap(j)$.
- For every server j: $\sum_i x(i,j) \le MaxServices(j)$.
- Latency SLA: if service i is assigned to j, latency(i,j) ≤ MaxLatency(i). An SLA exceedance makes the whole assignment infeasible, regardless of its objective score.

Performance measures

- Busy time of server j: $B(j) = \sum_i [Work(i) / Speed(j)] \cdot x(i,j)$, measured in seconds.
- Simulated makespan: M = max_j B(j), in seconds. An empty server has B(j)=0.
- CPU utilization fraction: $U(j) = [\sum_i CPU(i) \cdot x(i,j)] / CPUcap(j)$.
- Active power: $P(j)=IdlePower(j)+[PeakPower(j)-IdlePower(j)] \cdot U(j)$ when B(j)>0; otherwise P(j)=0 (assume an unused server is powered off).
- Energy consumed: $E = \sum_j [P(j) \cdot B(j)/3600]$, measured in watt-hours (Wh).
- Operating cost: $C = \sum_j [Price(j) \cdot B(j)]$, measured in USD.
- Per-service network latency: latency(i,j) = RTT(j) + 1000 × [8 × PayloadMB(i) / BandwidthMbPerSec(j)], measured in milliseconds.
- Mean service latency: $L = (1/12) \times \sum_i \sum_j [latency(i,j) \cdot x(i,j)]$ in ms. Worst-case latency: $Lmax = \max_i \sum_j [latency(i,j) \cdot x(i,j)]$.

Objective: minimize J = 0.35·(M / 30 s) + 0.20·(C / 0.50 USD) + 0.15·(E / 5 Wh) + 0.30·(L / 70 ms).

The reference scales (30 s, 0.50 USD, 5 Wh, 70 ms) and four weights are FIXED for the default benchmark and sum to 1.0. Values are benchmark normalizers, not hard limits. After data edits, use the same formula unless advanced settings explicitly change weights. The network delay approximates communication only; it does not include CPU service time or wait time. Never accept an SLA-violating candidate even if J is small.

5. Algorithm requirements

- Implement at least ONE genuine optimization method: Genetic Algorithm (recommended), Simulated Annealing, Ant Colony Optimization, a suitable discrete PSO, another approved metaheuristic, or an exact method with demonstrable search progress.
- Encode each candidate as a 12-element vector. Each element holds a server identifier (A, B, C or D). For a GA, describe population initialization, selection, crossover, mutation, survivor selection and elitism.
- Use constraint-aware generation, repair and/or rejection to handle infeasible candidates. Explain what happens when a candidate cannot be repaired.
- Keep the best feasible solution seen so far. Report best objective J, M, C, E, mean latency L, worst-case latency Lmax, SLA violations, iteration and algorithm wall-clock run time.
- Provide configurable iteration limit, population size (if applicable), seed, and at least two algorithm-specific parameters.
- Provide a simple comparator such as random-feasible search or First-Fit; compare results under the same objective and dataset. A single random/First-Fit allocation alone does not meet the algorithm requirement.
- The application must stop gracefully and give a clear reason if no feasible allocation is found. A failed search alone is not proof of mathematical infeasibility.

6. Required software interface and visualization

The visual interface must reflect real intermediate states of the running algorithm; an animation of only the final assignment is insufficient.

- **Configuration screen:** Editable servers/services: capacity, throughput, energy, price, RTT, bandwidth, input payload and maximum allowed latency; defaults/reset and validation.
- **Algorithm controls:** Start, pause, resume, stop, step one iteration, reset; adjustable seed and parameters; iteration counter and wall-clock time.
- **Live assignment canvas:** Show 12 services, four server panels, and animated assignment links for the current and best-so-far states; label each service network latency (ms).
- **Resource utilization:** For each server show CPU, RAM, disk, GPU and service-slot usage with exact numbers and progress bars, e.g. 14/16 vCPU. Show constraint violations in red and feasible status in green.
- **Progress chart:** Graph best J and mean latency L (ms) versus iteration; show at least two of makespan, energy and cost as additional series/panels.
- **Network/SLA view:** Show RTT, bandwidth and individual service transmission delays; display mean latency, worst latency, and SLA status/margin per service. Mark rejected assignments with a reason.
- **Execution timeline:** Display each server's ordered services as a Gantt-style timeline using Work/Speed processing durations; label server busy times and makespan.
- **Results & explanation:** Final assignment table, J/M/C/E/L/Lmax, SLA pass count (12/12), improvement vs. baseline, seed, run duration, feasibility and iteration log.

Usability: use readable labels and units; use more than color alone to communicate constraint status; provide clear empty/no-solution/error states. The UI may be built with JavaFX, or with a Java optimization core and a separate web frontend.

7. Verification and required test scenarios

Students must demonstrate the following tests and include screenshots, logs or automated assertions in the submission.

Test	Input / action	Expected behavior
T1	Run the default benchmark	Return an allocation, or report a bounded search failure; every accepted solution is feasible.
T2	Assign S06 to A deliberately	Reject: S06 requires 1 GPU and A has 0.
T3	Force S04 onto A already hosting S01,S03,S07	Reject: A would demand 21 vCPU > 16.
T4	Disable C GPU capacity (set C GPU slots=0)	Report no feasible placement: 3 GPU slots are demanded but only 1 exists.
T5	Set a demand exactly equal to a server limit	Accept equality: ≤ is valid.
T6	Run twice using identical data, seed and settings	Produce identical optimization results when using a deterministic seeded implementation.
T7	Set speed=0 or negative capacity	Reject invalid configuration before running; show helpful message.
T8	Compare timeline totals to computed B(j)	Timeline busy times and makespan match mathematical formulas.
T9	Force S03 onto C while keeping its 60-ms SLA	Reject: 60 + 1000×(8×1.5/1200) = 70 ms > 60 ms.
T10	Set server B bandwidth to 0 Mb/s or RTT to -1 ms	Reject invalid network configuration before optimization.

Test	Input / action	Expected behavior
T11	Decrease A bandwidth with assignment held fixed	S01/S03/S07 latency rises; mean L and J change consistently.
T12	Check reference mapping network values	All 12 SLAs pass and L matches 53.347863 ms (rounded).

8. Reference allocation for numeric self-check (valid, not optimal)

Server	Assigned services	CPU	RAM	Disk	GPU	Busy time (s)
A	S01, S03, S07	14/16	28/32	125/160	0/0	28.3333
B	S12	5/24	10/48	45/250	0/0	7.2500
C	S02, S05, S06, S10, S11	27/32	54/64	240/400	2/2	20.0000
D	S04, S08, S09	17/20	34/40	155/240	1/1	18.3333

Expected reference values (rounded): M = 28.3333 s; C = $0.449583; E = 3.927416 Wh; L = 53.347863 ms; Lmax = 76.666667 ms; all 12 latency SLAs pass; J = 0.856845. This is a feasible numerical self-check, not a proven optimum.

Latency spot checks: S03 on A = 18 + 1000×(8×1.5/500) = 42.000 ms (SLA 60); S11 on C = 60 + 1000×(8×2.5/1200) = 76.667 ms (SLA 105); S09 on D = 42 + 1000×(8×1.0/650) = 54.308 ms (SLA 80).

Percentage improvement over reference = 100 × (0.856845 − J_best) / 0.856845. Do not report a positive improvement if the candidate is infeasible.

9. Experiments and analysis

- Run the default instance at least 10 times with 10 different documented random seeds. Record J, M, C, E, L, Lmax, SLA violations, algorithm execution time and whether a valid solution was obtained.
- Report best and mean objective value, sample standard deviation, successful-run count and mean wall-clock time.
- Investigate the effect of a search-budget parameter (e.g. 20, 50, 100 and 200 generations/iterations), keeping other settings fixed; chart objective improvement against iteration count.
- Compare against a transparent baseline using the same J, hard resource constraints and per-service latency SLAs.
- Network sensitivity experiment: increase all RTTs by 25% or reduce each bandwidth by 30%, run with the same seeds and compare solution feasibility, J, L and Lmax.
- Explain trade-offs among server load, cost, energy, makespan and communication latency; discuss how GPU limitations and SLA thresholds restrict placement.

10. Implementation and submission

- Use Java 17 or later for the optimization engine. JavaFX or a separate web UI is acceptable. External charting/UI libraries are allowed, but the assigned optimization method must be implemented and explained by the student.
- Submit the complete runnable source code, dependency/build files and clear run instructions. No API key or paid infrastructure should be necessary.
- Provide a short report (recommended 5–8 pages): problem model, candidate representation, algorithm/pseudocode, feasibility handling, screenshots, tests, experiment table and discussion of limitations.
- Provide a 3–5 minute screen recording or live demonstration showing configuration, intermediate algorithm progress, final charts and the invalid-input/infeasible-case handling.
- List external libraries, references, any code reused and any AI assistance, consistent with the institution’s academic-integrity requirements.

11. Marking rubric (100 marks)

Criterion	Marks	Evidence expected
Correct model, latency & constraints	20	Resource and latency-SLA feasibility, correct J/M/C/E/L formulas, units and reference checks.
Optimization algorithm	25	Appropriate method, working iterative improvement, parameters, repair/rejection, reproducibility.
Live visualization and UX	20	Current vs best animation, resource/SLA panels, latency and fitness charts, timeline, controls.
Experimental evaluation	15	Baseline, 10 seeded trials, budget and network-latency sensitivity, charts, interpretation.
Software quality & test coverage	10	Modular code, input validation, deterministic tests and graceful failures.
Documentation & demonstration	10	Clear report, instructions, explainable demo and properly credited tools.
TOTAL	100	

Optional bonus (+5 maximum, at instructor discretion): prove the optimum on a reduced instance using exhaustive enumeration; implement a second algorithm with fair comparison; or export a complete experiment report with interactive replay. Bonus cannot replace missing mandatory requirements.

12. Acceptance checklist

☐ All 12 services are assigned exactly once, with zero violated constraints.

☐ Four server network profiles, 12 service payloads and SLA limits are editable alongside all resource inputs.

☐ The optimization algorithm actually searches, improves or meaningfully explores multiple candidates; its steps can be replayed or viewed live.

☐ The current candidate and best-so-far assignment, utilization bars and objective chart are linked to the same real algorithm state.

☐ The final displayed J, M, C, E, L and Lmax match formulas/reference values; all 12 latency SLAs pass.

☐ The software handles invalid inputs, infeasible datasets, stop/pause and repeatable seeded runs.

☐ Source code, reproducible experiments, report and demo are supplied.

Instructor note

The assessment is intentionally fixed to 12 services and 4 servers. Extra servers, service-to-service traffic, stochastic congestion, real cloud deployment and monitoring are outside mandatory scope. A metaheuristic must present its result as “best found”; only an exact verification procedure may label a solution “globally optimal.”